

Entidades (Rectángulos)

Estas son los elementos principales para guardar:

- Temporada: Representa un año de campeonato
- Gran Premio: Un evento de carrera específico.
- Circuito: La pista física donde ocurre la carrera.
- Piloto: Los pilotos que compiten.
- Escudería: Los equipos o constructores.
- Vuelta: Representa una vuelta individual.
- Boxes: Representa una parada en boxes.
- Clasificación: Representa el resultado de la clasificación.

Atributos (Óvalos)

Estos son los datos que describen a cada entidad. Como:

Para Piloto: Tenemos PilotoRef (su ID único), Nombre, Apellido, F. de Nac. (Fecha de Nacimiento), Nacionalidad, etc.

Para Circuito: Tenemos CircuitoRef (su ID), Nombre, País, Ciudad y Coordenadas

Atributo Compuesto: Coordenadas es un atributo compuesto, se divide en más datos: Latitud, Longitud y Altura.

Relaciones y Cardinalidad (1:N, N:M)

Las relaciones definen las reglas de negocio y cómo se conectan las tablas.

Temporada (1) Inscrito en (N) Gran Premio

Significado: Una Temporada (1) tiene muchos (N) Grandes Premios. (Relación 1 a N).

Circuito (1) Donde (1.N) Gran Premio

Significado: Un Circuito (1) es la sede de uno o muchos (1.N) Grandes Premios (a lo largo de diferentes años). (Relación 1 a N).

Piloto (N) Corre (M) Gran Premio

Significado: Esta es una relación de Muchos a Muchos (N:M). Un Piloto (N) Corre en muchos Grandes Premios (M), y un Gran Premio tiene muchos Pilotos corriendo.

Entidad Asociativa: Cuando una relación N:M tiene sus propios atributos esa relación se convierte en su propia tabla en la base de datos final. En nuestro script, Corre es la tabla pl1final.results y tiene su propia tabla en el modelo relacional.

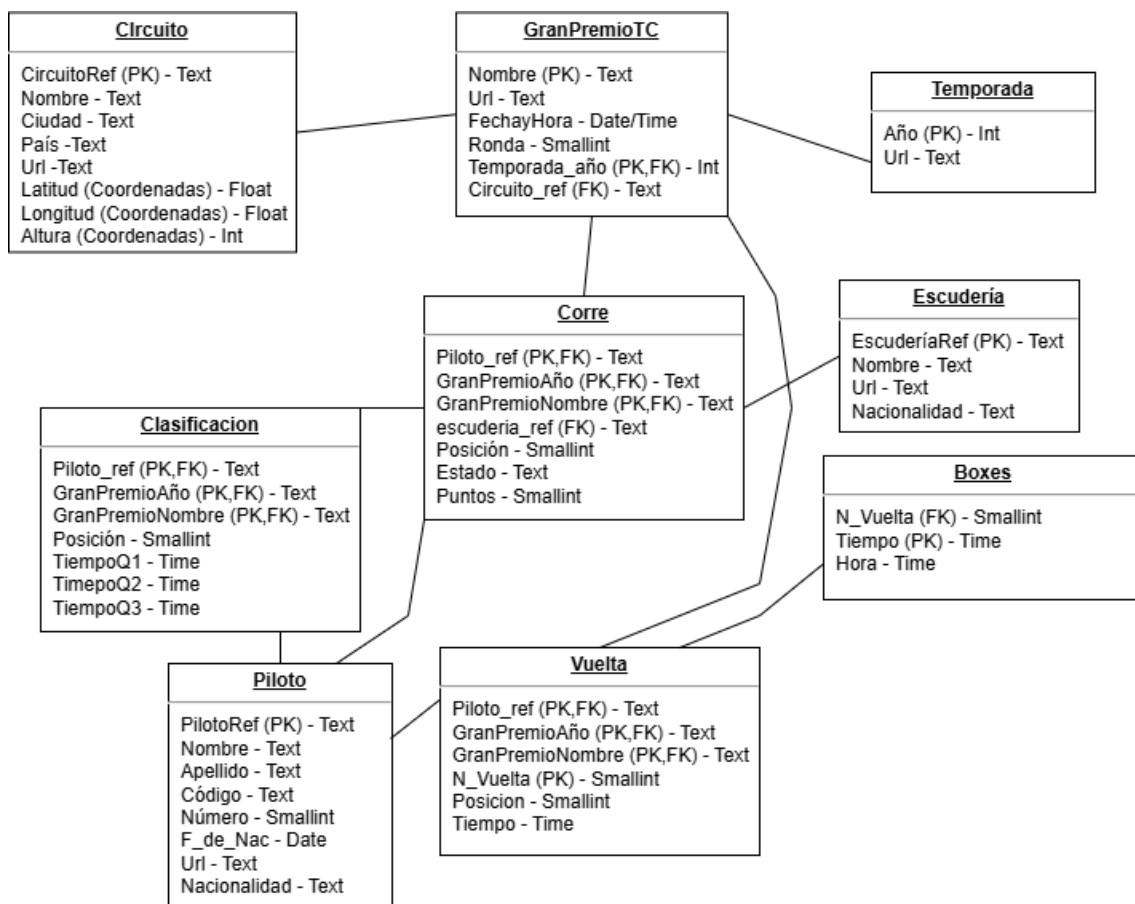
Otras Relaciones N:M que se convierten en Tablas:

Clasificación: Conecta Piloto (N) y Gran Premio (M). Se convierte en la tabla pl1final.qualifying y tiene su propia tabla en el modelo relacional.

Vuelta: Conecta Piloto (N) y Gran Premio (M). Se convierte en la tabla pl1final.lap_times y tiene su propia tabla en el modelo relacional.

Boxes: Conecta Piloto (N) y Gran Premio (M). Se convierte en la tabla pl1final.pit_stops y tiene su propia tabla en el modelo relacional.

Modelo Relacional:



El Modelo Relacional es el plano de la base de datos.

Es una traducción directa del MER a una estructura lógica que la base de datos puede entender. En este modelo, ocurren las siguientes modificaciones:

Las Entidades se convierten en Tablas.

Los Atributos se convierten en Columnas.

En este modelo se definen los tipos de datos.

Las Relaciones se implementan mediante Claves.

Las relaciones (1:N, N:M) son reemplazadas por Claves Primarias (PK) y Claves Foráneas (FK), que son las que crean las conexiones.

Componentes de Nuestro Modelo Relacional

Nuestro modelo se divide en dos tipos de tablas, que se pueden ver claramente en el diagrama:

Tablas de Dimensión

Estas tablas son las listas principales. Son las tablas a las que apuntan las demás.

Temporada: Almacena los años. Su Clave Primaria (PK) es Año.

Circuito: Almacena las pistas. Su PK es CircuitoRef.

Escudería: Almacena los equipos. Su PK es EscuderiaRef.

Piloto: Almacena los pilotos. Su PK es PilotoRef.

Tablas de Hechos

Estas tablas registran los eventos. Su trabajo principal es conectar las tablas de dimensión entre sí.

GranPremioTC:

Es la tabla de hechos central. Conecta Temporada y Circuito.

Su Clave Primaria (PK) es compuesta: (Nombre, Año). Esto garantiza que solo pueda existir un "Gran Premio de Mónaco" en 2023.

Tablas de Resolución N:M:

Corre: Es el mejor ejemplo. Nace de la relación N:M entre Piloto y GranPremio.

Clasificacion: Conecta Piloto y GranPremio.

Vuelta: Conecta Piloto y GranPremio.

Boxes: Conecta Piloto y GranPremio.

Claves (PK y FK)

PK (Primary Key / Clave Primaria):

Es el identificador único de una tabla.

FK (Foreign Key / Clave Foránea):

Conecta una tabla de hechos con una tabla de dimensión.

Código SQL

1. Introducción

Este código es un proceso completo una base de datos. Su objetivo es coger 10 archivos CSV sobre carreras de Fórmula 1 y:

1. Cargar los datos sucios en un esquema temporal (PL1temp).
2. Limpiar, convertir y conectar los datos (ej. convertir texto a números, manejar valores nulos, conectar tablas).
3. Mover los datos limpios y estructurados a un esquema final y relacional (pl1final) listo para ser consultado.

2. Estructura del código

El código está dividido en 3 fases principales, diseñadas para ejecutarse en orden.

1: Creación y Carga Temporal (PL1temp)

- **BEGIN:** Inicia una transacción. La transacción se cancela al final del código (ROLLBACK) y la base de datos queda intacta.

```
2 | BEGIN;  
3 |
```

La fase de carga.

1. **CREATE SCHEMA PL1temp:** Crea un esquema temporal para los datos.

```
8 | -- 'PL1temp' es la "carpetita" temporal para meter los datos sucios del CSV.  
9 | CREATE SCHEMA IF NOT EXISTS PL1temp;
```

2. **CREATE TABLE PL1temp.*_temp:** Crea 10 tablas (una por cada CSV).
 - **Punto Clave:** Todas las columnas se definen como TEXT para asegurar que la importación no falle.

```

11 -- Crea la tabla temporal de circuitos.
12 -- Todo como TEXTO, da igual, es un borrador.
13 CREATE TABLE PL1temp.circuits_temp (
14     circuitId TEXT,
15     circuitRef TEXT,
16     name TEXT,
17     location TEXT,
18     country TEXT,
19     lat TEXT,
20     lng TEXT,
21     alt TEXT,
22     url TEXT
23 );

```

3. **\copy:** Este es el comando que lee los CSV del disco duro y los mete en las tablas temporales.
 - **HEADER:** Hace que ignore las cabeceras.
 - **DELIMITER ',':** Especifica que los datos están separados por comas.
 - **NULL '\N':** Esta es la primera capa de limpieza. Le dice a \copy que, si encuentra el texto \N en el CSV, lo debe interpretar como un verdadero NULL (ausencia de dato) en la base de datos.

```

141 -- Texto \N en el CSV es un valor NULO.
142
143 \copy PL1temp.circuits_temp from 'C:\Users\dmfer\Downloads\Nueva
    carpeta\circuits.csv' WITH (FORMAT csv, HEADER, DELIMITER ',', NULL '\N', ENCODING
    'UTF-8');
144 \copy PL1temp.constructors_temp from 'C:\Users\dmfer\Downloads\Nueva
    carpeta\constructors.csv' WITH (FORMAT csv, HEADER, DELIMITER ',', NULL '\N',
    ENCODING 'UTF-8');

```

2: Creación del Esquema Final (pl1final)

Se crea la estructura final.

1. **CREATE SCHEMA pl1final:** Crea el esquema final para los datos.

```

159 -- Crea el esquema 'pl1final' donde estarán las tablas limpias, normalizadas y con
    tipos de datos correctos.
160 CREATE SCHEMA IF NOT EXISTS pl1final;

```

2. **Tablas Independientes:** Se crean primero las tablas Padre (seasons, circuits, constructors, drivers, status).
 - Usan tipos de datos correctos (ej. INT, FLOAT, DATE).
 - Definen CONSTRAINT y PRIMARY KEY para evitar duplicados.

```

166 CREATE TABLE pl1final.seasons (
167     -- Ahora se usan tipos de datos específicos (INT) en lugar de TEXT.
168     year INT, url TEXT,
169     -- 'CONSTRAINT ... PRIMARY KEY': Define la Clave Primaria (PK) de la tabla.
170     -- Esto impone una restricción (no puede haber dos filas con el mismo 'year')
171     -- y permite que otras tablas (como 'races') se conecten a esta.
172     CONSTRAINT seasons_pk PRIMARY KEY (year)
173 );
174 CREATE TABLE pl1final.circuits(
175     circuitRef TEXT NOT NULL, name TEXT, location TEXT, country TEXT,
176     -- Se usa 'FLOAT' para permitir números con decimales.
177     lat FLOAT, lng FLOAT, alt INT, url TEXT,
178     -- 'circuitRef' (el código del circuito) se usa como PK.
179     CONSTRAINT circuits_pk PRIMARY KEY (circuitRef)
180 );

```

3. **Tablas Dependientes:** Se crean después las tablas Hijo (races, lap_times, pit_stops, qualifying, results).
- **CONSTRAINT y FOREIGN KEY:** Crean las conexiones que garantizan los datos.
 - **REFERENCES pl1final.seasons(year):** Esta es la regla que conecta a la hija con el padre. Asegura que no se pueda insertar una carrera para un año que no exista.
 - **ON DELETE RESTRICT:** Regla de seguridad. Impide borrar un padre si un hijo todavía lo está usando.
 - **ON UPDATE CASCADE:** Regla de sincronización. Si la clave de la tabla padre cambia, todas las tablas hijo se actualizan automáticamente.

```

201 CREATE TABLE pl1final.races (
202     year INT NOT NULL, round INT, name TEXT NOT NULL, circuitRef TEXT, date_time
    TIMESTAMP, url TEXT,
203     CONSTRAINT races_pk PRIMARY KEY (year, name),
204     CONSTRAINT races_seasons_fk FOREIGN KEY (year) REFERENCES pl1final.seasons
    (year) ON DELETE RESTRICT ON UPDATE CASCADE,
205     CONSTRAINT races_circuits_fk FOREIGN KEY (circuitRef) REFERENCES
    pl1final.circuits (circuitRef) ON DELETE RESTRICT ON UPDATE CASCADE
206 );

```

3: Transformación y Carga (INSERT INTO, SELECT)

Esta es la fase de limpieza y movimiento de datos.

- **Patrón General:** INSERT INTO [tabla_limpia] SELECT [datos_limpiados] FROM [tabla_sucia];

```

241 INSERT INTO pl1final.seasons (year, url)
242 SELECT DISTINCT ON (year)
243     year::INT, url
244 FROM PL1temp.seasons_temp;
245

```

- **Limpieza de Nulos (Transformación 1):**
 - **NULLIF(lat, '')::FLOAT:**

- Los CSV tienen dos tipos de nulos: \N (que \copy ya convirtió en NULL) y "" (strings vacíos).
- Este comando soluciona el problema de los strings vacíos.
- **FLOAT**: Convierte el resultado al tipo FLOAT. Esto evita el error de sintaxis no válida.

```

248 SELECT DISTINCT ON (circuitRef)
249     circuitRef, name, location, country,
250     NULLIF(lat, '\N')::FLOAT,
251     NULLIF(lng, '\N')::FLOAT,
252     NULLIF(alt, '\N')::INT,
253     url
254 FROM PL1temp.circuits_temp;

```

- **Conversión de Tipos (Transformación 2):**

- **year::INT**: Convierte el texto "2023" en el número 2023.
- **TO_TIMESTAMP(...)**: Una transformación que coge dos columnas de texto (date y time), las une con un espacio, y las convierte en un único campo de tipo TIMESTAMP (fecha y hora).

```

278 INSERT INTO pl1final.races (year, round, name, circuitRef, date_time, url)
279 SELECT
280     tr.year::INT, NULLIF(tr.round, '\N')::INT, tr.name, tc.circuitRef,
281     TO_TIMESTAMP(NULLIF(tr.date, '\N') || ' ' || NULLIF(tr.time, '\N'), 'YYYY-MM-DD
HH24:MI:SS'),
282     tr.url
283 FROM PL1temp.races_temp AS tr

```

- **Traducción de IDs (JOINS):**

- En INSERT INTO pl1final.races se usa LEFT JOIN PL1temp.circuits_temp AS tc.
- **JOIN**: Se usa para "traducir" los IDs temporales (ej. circuitId = 5) a las claves de referencia finales (ej. circuitRef = 'monza').
- **LEFT JOIN**: Se usa para circuits y status. Significa "intenta encontrar el circuito; si no lo encuentras, no te preocupes, inserta la carrera con el circuito en NULL".
- **JOIN**: Se usa para races, drivers, y constructors en la tabla results. Significa "el resultado DEBE tener una carrera y un piloto. Si no los encuentras, descarta este dato."

```

284 LEFT JOIN PL1temp.circuits_temp AS tc ON tr.circuitId = tc.circuitId;
285 --El "LEFT" lo que dice esq si se puede haber más de una carrera con el mismo
circuitRef
286 --Se han corrido más carreras en el mismo circuito

```

- **Manejo de Duplicados (SELECT DISTINCT ON):**

- Se usa en `pl1final.results` para manejar los datos duplicados del CSV (ej. el piloto 'ertl').
- **SELECT DISTINCT ON (tr.name, tr.year, td.driverRef)**: Le dice a PostgreSQL "Solo quiero una fila por esta combinación (la clave primaria)".
- **ORDER BY tres.resultados_id ASC**: Es la regla de desempate obligatoria. Le dice a DISTINCT ON cuál de los duplicados elegir: el que tenga el `resultados_id` más bajo (es decir, el primero que apareció en el CSV).

```
336 ORDER BY tr.name, tr.year, td.driverRef, tres.resultados_id ASC;
337 --"Order By" por si hay duplicados solo quedarme con uno y en caso de empate usar
    resultados id en ASC(forma ascendente)
338 --Los demás los descarto
```

4: Finalizar la Transacción

- **ROLLBACK**: El comando final. La base de datos habría ejecutado un ROLLBACK deshaciendo todo.

```
342 ROLLBACK;
343
```